

3.1 Introduction to Memory Unit

A **memory unit** is a part of a microcomputer, and its **main job** is to **store instructions and data**. Memory should be **fast enough** to match the speed of the microprocessor. The problem is that **fast memory is expensive**, and large, fast memory is **not affordable using a single technology**. The solution is to **use different technologies** to balance **speed and cost**, such as **solid-state memory, which is fast, magnetic memory, which is cheaper but slower, and optical memory, like CDs or DVDs**. Types of Memory in a Microcomputer

- Microprocessor memory (Registers)
- Main memory (Primary memory)
- Secondary memory

Microprocessor Memory (Registers)

Registers are **small storage locations inside the CPU** used to store **temporary results during calculations**. They are **very fast because they are made with the same technology as the processor**, but they are **expensive**, so only a few registers are used. For example, a **CPU may have 8 or 16 general-purpose registers** to hold temporary data and results while performing calculations.

Main Memory (Primary Memory)

Main memory stores all **programs and data that are currently running**, and the CPU can **access this memory directly**. It is usually made using **CMOS technology**, which makes it larger than processor memory but **slower**. Main memory includes **RAM**, which is **volatile and loses data when power is off**, and **ROM**, which is **non-volatile and retains data even when power is off**. For example, a **Pentium CPU uses 32-bit addresses to access up to 4 GB of memory**, and some CPUs divide memory into segments, like **Pentium real mode uses 1 MB memory divided into 16 segments of 64 KB each**, whereas the **Motorola 68000 CPU uses linear memory with no segmentation**.

Secondary Memory

Secondary memory is used to store **large programs and data at low cost**, such as **hard disks, CDs, and DVDs**, and is **slower than main memory**. Memory in disks is organized into **tracks**, which are concentric rings on the disk surface, and **sectors**, which are subdivisions of tracks and usually contain **512 or 1024 bytes**. For example, **CDs use an infrared laser for reading**, while **DVDs use a red laser with a shorter wavelength, allowing more storage**.

3.2 Main Memory

Main memory stores **both instructions and data**, and the **size of each memory word depends on the CPU type**, such as **8-bit CPUs where 1 word = 1 byte** and **16-bit CPUs where 1 word = 2 bytes**. Each memory word has an **address used by the CPU to access data**, for instance, **Pentium uses 32-bit addresses to access 4 GB of memory**. Memory can be **volatile**, like **RAM**, which loses data if power is off, or **non-volatile**, like **ROM**, which retains data even when power is off. Some CPUs use **memory segmentation**, for example, **Pentium real mode divides 1 MB memory into 16 segments of 64 KB each**, while **Motorola 68000 uses linear memory**.

Types of Main Memory

- ROM (Read-Only Memory)
- RAM (Random Access Memory)

3.2.1 Read-Only Memory (ROM)

ROM, or **Read-Only Memory**, is a type of memory that can **only be read and not written**. It is **non-volatile**, which means it **retains data even when power is off**, and is usually made using **CMOS technology**. There are several types of ROM.

Types of ROM

- **Mask ROM** → programmed during manufacturing and is **permanent**, so it **cannot be changed**.
- **EPROM (Erasable Programmable ROM)** → can be programmed and erased using special equipment called an **EPROM programmer**, and is **erased using UV light for 10–20 minutes**. The chip must be **removed from the system** to erase or program.
- **EAROM / EEPROM (Electrically Alterable/Erased ROM)** → can be **programmed or erased without removing the chip**, and is considered **read-mostly memory**, meaning **reading is faster than writing**.
- **Flash Memory** → combines **EPROM and EEPROM features**, can **reprogram many bytes at once electrically**, and is used in devices like **cell phones and digital cameras**. An example is **Intel 28F020**.

3.2.2 Random-Access Memory (RAM)

RAM is **temporary memory** used to store data and instructions while a program is running. It is **volatile**, meaning it **loses data when power is off**. RAM is divided into several types.

Types of RAM

- **Static RAM (SRAM)** → stores data in **flip-flops**, **does not need refresh**, is **faster and more expensive**, and has **larger cell size**.
- **Dynamic RAM (DRAM)** → stores data in **capacitors**, **must be refreshed frequently (every few milliseconds)**, is **slower and cheaper**, and has **smaller size**. Examples include **4464, 44256, and 41000 DRAM chips**.

Enhanced DRAM

- **EDO DRAM (Extended Data Output)** → outputs the next data while the current data is being read, making it **faster than normal DRAM**.

- **SDRAM (Synchronous DRAM)** → works in **sync with the system clock**, allowing **faster and higher-density memory**. It usually combines **multiple DRAM chips in one package** and is widely used in **personal computers**.

3.2.3 Memory READ and WRITE Operations

When the **microprocessor executes instructions**, it performs two main steps: **Instruction Fetch**, which gets the instruction from memory, and **Instruction Execution**, which performs the operation.

Instruction Fetch works like this: the CPU places the **program counter (PC) value on the address bus**, sets the **READ pin HIGH** to tell memory to send the instruction, memory sends the **instruction via the data bus**, and the CPU stores it in the **instruction register** ready for execution.

Memory READ is when the CPU **gets data from memory into its register**. The steps are: the CPU fetches the instruction and decides it's a READ operation, puts the **memory address on the address pins**, sets **READ HIGH**, memory sends the **data to the CPU**, and the CPU stores the **data in an internal register**.

Memory WRITE is when the CPU **stores its data into memory**. The steps are: the CPU fetches the instruction and decides it's a WRITE operation, puts the **memory address on the address pins**, sets **WRITE HIGH**, CPU sends the **data on the data bus**, and memory stores the **data at the given address**.

- **ROM** → permanent memory; types: Mask ROM, EPROM, EEPROM, Flash (examples: Intel 28F020, cell phones, digital cameras)
- **RAM** → temporary memory; types: SRAM (fast, expensive, flip-flops), DRAM (slow, cheap, capacitors, examples: 4464, 44256, 41000)
- **EDO DRAM & SDRAM** → faster versions of DRAM used in PCs
- **Instruction Fetch** → memory READ to get instruction
- **Memory READ** → CPU reads data from memory
- **Memory WRITE** → CPU writes data to memory

3.2.4 Main Memory Organization

Main memory consists of **ROM and RAM**. RAM is more complex than ROM because it supports both **read and write operations**. Main memory organization refers to **how ROMs and RAMs are connected together** and **how memory locations are assigned**. Proper memory organization helps the designer place **permanent programs in ROM**, **temporary programs in RAM**, and use RAM for **subroutines and stacks during interrupts**.

Dynamic RAM (DRAM)

Dynamic RAM stores data in **MOS capacitors**, which require **periodic refreshing** to retain data. DRAM is **cheaper than SRAM**, has **high memory density**, and consumes **low power**, so it is commonly used when memory size is **16K words or more**. DRAM uses a **row and column addressing method**. For example, a **1 Mb DRAM** requires **20 address bits**, where **10 bits are**

used for the row address and 10 bits for the column address, and it uses control lines called **RAS (Row Address Strobe)** and **CAS (Column Address Strobe)**.

Expanding DRAM

DRAM can be expanded easily because adding **one additional address line** doubles both the number of rows and columns, resulting in a **four times increase in memory size**. A **DRAM controller** is used to manage the memory system. It performs **automatic refreshing**, usually every **4 milliseconds**, and generates a **wait signal** if the CPU attempts to access memory during the refresh cycle.

Buffers and Multiplexers

When memory size becomes large, **buffer circuits** are required to provide strong address and data signals. Address buffering is done using **74LS244 or 74HC244**, while data buffering is done using **74LS245 or 74HC245**. **Multiplexers**, such as **74LS157 or 74HC157**, are used to divide the CPU address lines into **row and column addresses** required by DRAM.

3.2.5 Main Memory Array Design

Main memory array design means connecting **multiple memory chips together** to form the required memory size. This arrangement allows the memory system to **interface properly with the microprocessor** while achieving the desired capacity.

Interfacing Memory with CPU

A microprocessor with **16 address lines (A0–A15)** can address $2^{16} = 64\text{K}$ bytes of memory. The microprocessor uses control signals such as **M/IO**, which is **LOW for I/O instructions and HIGH for memory instructions**, **READ**, which is HIGH during read operations, and **WRITE**, which is HIGH during write operations. The **data lines (D0–D7)** are **bidirectional**, allowing data transfer between CPU and memory, while **address and control lines** are **unidirectional**, carrying signals from the CPU to memory.

SRAM Memory Chip Example (1K × 8)

A **1K × 8 SRAM** uses **10 address lines (A0–A9)**, which can address $2^{10} = 1024$ memory words. It has **8 bidirectional data lines (D0–D7)** that allow the CPU to read and write data. The SRAM chip uses control signals **m**, **CS2**, and **WE**. The SRAM is enabled when **m = 0 and CS2 = 1**. When **WE = 0**, the memory performs a **write operation**, and when **WE = 1**, the memory performs a **read operation**.

Quick Exam Notes

Memory Organization

- Main memory = ROM + RAM
- RAM is more complex because it supports read and write
- DRAM uses row/column addressing and requires refreshing

- SRAM is faster and simpler and used in smaller memory arrays

Memory Array Design

- Multiple memory chips are connected to form required memory size
- CPU communicates using address and control lines
- Data lines are bidirectional, address lines are unidirectional

SRAM Example

- 1K × 8 RAM contains 1024 memory words
- Enabled when $m = 0$ and $CS2 = 1$
- $WE = 0$ for write, $WE = 1$ for read

Memory Decoding in Microprocessor Systems

When connecting a **microprocessor** to **ROM/RAM chips**, you need to decide **which chip is selected for a particular address**. This is called **address decoding**.

Two common techniques:

1. **Linear (or Direct) Decoding**
2. **Full (or Complete) Decoding**

1. Linear (Direct) Decoding

How it works

- Connect **all address lines (A0–An)** from CPU to all memory chips.
- Connect **control lines (READ/WRITE)** from CPU to chips.
- **High-order address bits** act as **chip selects**.
 - Example: $4 \times 1K$ SRAM $\rightarrow$ 4K total memory
 - A10, A11 $\rightarrow$ select chips 1–4
 - Lower bits A0–A9 $\rightarrow$ select memory location inside the chip

Advantages

- **No extra hardware (decoder) needed** $\rightarrow$ simpler circuit

Disadvantages

1. **Bus conflicts** → if two high-order bits are LOW at the same time, **two chips may be selected** → causes errors
2. **Wasted address space** → some addresses are mirrors of others (called **memory foldback**)

Example (Address Map for 4 × 1K SRAM):

Address Range (Hex)	SRAM Chip
3800–3BFF	I
3400–37FF	II
2C00–2FFF	III
1000–1FFF	IV

- Can expand up to 6K with extra address bits
- Mostly used in **small systems**

2. Full (Complete) Decoding

How it works

- Use a **decoder** (like 74LS138) to select **exactly one chip** based on **high-order address bits**.
- Decoder output → connected to **chip select** of memory chips.
- Only **one chip** is selected at a time → **no bus conflicts**.

Advantages

1. **No wasted addresses** → unused outputs can be used for **expansion**
2. **No bus conflicts** → safe to use

Example

- 4 × 1K SRAM with **3-to-8 decoder**:
 - High-order address bits: A12, A11, A10 → input to decoder
 - Only **one output active** → enables only one SRAM chip

Modern Note

- **FPGAs** can now perform memory decoding for **32-bit CPUs** (like Intel Pentium, 68020)
- Faster and more flexible than hardware decoders

3.3 Microprocessor On-Chip Memory Management Unit (MMU) and Cache

Many **32-bit CPUs** (Pentium, 68030, 68040) have **on-chip MMU and cache memory**.

3.3.1 Memory Management Concepts

Why Memory Management is Needed

- Systems store **huge amounts of data** → usually on **hard disk (secondary memory)**.
- Accessing disk **every time** → **very slow**.
- Solution: Use **fast main memory (RAM)** locally.
- But **RAM is expensive**, so the system uses **both RAM + disk** efficiently.

Memory Management Tasks

The system must:

1. **Transfer blocks of data** between RAM and disk efficiently
2. **Track which blocks are used**
3. **Replace blocks intelligently**
4. **Allocate memory dynamically**

Memory Protection

- The **OS must protect memory** from users interfering with each other.
- Users can **share code/data** safely but cannot **overwrite others' areas**.
- This requires a **device between CPU and memory** → called **Memory Management Unit (MMU)**.

What is MMU?

- **MMU = Memory Management Unit** → manages memory access.
- **Functions of MMU:**
 1. **Address translation** → converts **logical addresses** (used in program) to **physical addresses** (actual memory).
 2. **Protection** → prevents illegal memory access.

Logical vs Physical Addresses

- **Logical address** = address used in program (symbolic).
- **Physical address** = actual address in RAM chips.

- **MMU translates logical → physical** using:
 1. **Substitution technique** → replace logical address with physical → **faster**
 2. **Offset technique** → add a number (offset) to logical address → **more flexible**

Example:

Method	Description
Substitution	Direct mapping of logical → physical
Offset	Logical + offset → physical

Memory Units

- Memory is divided into **small units**:
 3. **Pages** → equal-sized blocks (paging)
 4. **Segments** → variable-sized blocks (segmentation)
- **Paging** makes it easier to maintain **address translation tables**.

Mapping Logical → Physical Addresses

- **Paging** → divide memory into pages
- **Segmentation** → divide memory into segments
- **Combined paging + segmentation** → flexible system
- **Virtual memory**:
 - User can access **more memory than physically available**
 - **MMU hardware** → translates addresses
 - **OS software** → manages page replacement, swapping data between RAM and disk

MMU + OS Tasks Example:

1. Remove a page from RAM to make room for a new page
2. Load new page from disk to RAM
3. Place page in the correct location in memory

MMU (Memory Management Unit):

- Located **between CPU and memory**
- **Translates logical → physical addresses**
- **Protects memory from illegal access**

Address Translation Methods:

1. **Substitution** → fast, direct mapping
2. **Offset** → flexible, adds an offset to logical address

Memory Division:

- **Paging** → equal-sized pages
- **Segmentation** → variable-sized segments
- **Combined** → flexible mapping

Virtual Memory:

- User can access **more memory than physically present**
- **Hardware (MMU)** → address translation
- **Software (OS)** → page replacement, manage RAM/disk efficiently

Page Replacement and Memory Management

When **main memory (RAM)** is **full**, and a new page is needed from **secondary memory (disk)**:

- The system must **remove a page** from RAM → this is called **page replacement**.

Popular Page Replacement Policies

1. **FIFO (First In First Out)**

- Removes the page that has been in RAM **the longest**.
- Easy to implement.
- **Disadvantage:** May remove heavily used pages → bad if frequently used pages are replaced.
- **Example:** If a text editor is heavily used, FIFO may remove it → system slows down.

2. **LRU (Least Recently Used)**

- Removes the page that **has not been used for the longest time**.
- Better for frequently used pages.

Segmentation and Memory Management

- **Memory divided into segments** → variable-sized blocks.

- MMU uses a **segment selector** → finds a **descriptor table** entry:
 - Descriptor = physical base address + privilege level + control bits

How segmentation works

1. CPU gives **logical address** to MMU.
2. MMU checks if the segment is in **physical memory**.
3. If yes → adds **offset** to **segment base** → physical address
4. If no → interrupts CPU → OS loads segment from disk to RAM
5. Segments may consist of **multiple pages** (e.g., 256 bytes each).

Problem: External Fragmentation

- When memory has **small free areas between segments**, too small to load a new segment → unusable space
- MMU uses special techniques to handle it.
- **Advantage of segmentation:** few descriptors for large programs, reduces internal fragmentation
- **Disadvantages:** external fragmentation, complex placement algorithms, longer swap times

Protection Using MMU

- Each segment has a **privilege level**.
- CPU program also has a **privilege level**.
- MMU compares the two:
 - If CPU privilege $\geq$ segment privilege → **access allowed**
 - If CPU privilege $<$ segment privilege → **interrupt** → protects OS or critical data

Paging and Fragmentation

- **Paging** → divides memory into **equal-sized pages** → solves **external fragmentation**
- **Internal fragmentation:** leftover unused memory inside a page
 - Example: need 300 bytes, page size = 1 KB → 700 bytes unused

Paged-Segmentation

- Memory = **segments** → **divided into pages**

- **Logical address** = segment + page + word
 - **Segment** → which segment
 - **Page** → which page in the segment
 - **Word** → which word in the page
- **Segment size** = depends on number of pages assigned

Protection Mechanism

1. **Physical memory protection:** protection bits for each block
2. **Logical memory protection:** protection bits in **segment descriptor** → more efficient

Virtual Memory

- Allows programs to **use more memory than physically available**
- **Virtual address** = address used by program
- **Physical address** = actual RAM location
- MMU translates virtual → physical addresses
- Virtual memory allows:
 - Space allocation
 - Program relocation
 - Code sharing
 - Memory protection
- **Modern 32-bit CPUs (Pentium, 68030, 68040):**
 - Have **on-chip MMU** and **cache**
 - Virtual addresses first go to **cache**, then RAM if not present

Quick Exam Notes

Page Replacement Policies

- **FIFO:** oldest page removed → may remove heavily used pages
- **LRU:** least recently used page removed → better choice

Segmentation

- Memory divided into **segments (variable-sized)**

- MMU adds **offset to segment base** → physical address
- External fragmentation possible → gaps between segments

Paging

- Memory divided into **equal pages** → avoids external fragmentation
- Internal fragmentation may occur → leftover unused space in a page

Paged-Segmentation

- Each segment = multiple pages
- Logical address = **segment + page + word**

Protection

- MMU checks **privilege levels** → prevents illegal access

Virtual Memory

- Programs can use **more memory than physical RAM**
- MMU translates **virtual** → **physical addresses**
- Hardware + OS work together → page swapping, allocation, and protection

3.3.2 Cache Memory Organization

Why Cache Memory?

- **Purpose:** Improve microprocessor performance.
- **How:** Put a **small, very fast memory** between CPU and main memory.
- **Reason:** Programs often **access the same memory locations repeatedly** → called **locality of reference**.
- Example: Loops in programs often reuse the same instructions → cache helps fetch them faster.

Example CPUs with cache: Intel 80486/Pentium, Motorola 68030/68040.

- 80386 does **not** have on-chip cache → uses external cache.

How Cache Works

1. CPU requests data/instruction → checks **cache first**.
2. **Cache Hit:** Data found in cache → CPU reads it directly → very fast.

3. **Cache Miss:** Data **not in cache** → CPU fetches from main memory and also **stores it in cache**.

Cache size vs main memory:

- Cache = small but fast (100 ns)
- Main memory = large but slower (500 ns)

Block transfer:

- A block of 4–16 words is transferred from main memory to cache on a miss.
- Replacement policies: **FIFO** or **LRU**

Cache Mapping Techniques

Cache must decide **where to place main memory data**.

1. Direct Mapping

- Each main memory block maps to **one specific cache line**.
- Example:
 - Main memory = 4K words
 - Cache = 256 words
 - Memory address split → **index + tag**
 - Index → cache line to use
 - Tag → check if the correct block is in that line
- **Drawback:** Multiple addresses with same index → frequent misses.

2. Fully Associative Mapping

- Any memory block can go **anywhere in cache**
- Cache stores **both address and data**
- CPU compares requested address with **all cache addresses at once**
- **Very fast**, but **expensive**

3. Set-Associative Mapping

- Combination of direct & fully associative
- Cache divided into **sets**, each set can hold multiple blocks
- Faster than fully associative, cheaper than fully associative

Direct Mapping Example

- Memory address = 12 bits
- Cache address = 8 bits (256 words)
- 12-bit address → **4-bit tag + 8-bit index**
- CPU wants address 100 → index = 00 → compare tag with cache → miss → fetch from main memory → store in cache

Fully Associative Example

- Cache stores **both address and data**
- CPU searches **all cache entries simultaneously**
- If found → cache hit
- If not found → cache miss → fetch from main memory → write in cache
- Replacement policy → FIFO or LRU

Cache Types

1. **Instruction cache:** stores instructions only
2. **Data cache:** stores data only
3. **Unified cache:** stores both instructions and data

Quick Exam Notes – Cache Memory

- **Why cache?** Small, fast memory → stores frequently accessed data → speeds up CPU
- **Cache hit:** Data found → fast
- **Cache miss:** Data not found → fetch from main memory → update cache
- **Block transfer:** 4–16 words per transfer
- **Mapping techniques:**
 1. **Direct mapping:** simple, may cause frequent misses
 2. **Fully associative:** any block anywhere → fast but expensive
 3. **Set-associative:** compromise → faster + cheaper
- **Replacement policies:** FIFO / LRU

Set-Associative Cache Memory

- A mix of **direct mapping** and **fully associative mapping**.
- Each **cache word (line)** stores **multiple main memory words** → called a **set**.
- Each memory word in a set has:
 - **Tag** → identifies the block
 - **Data word** → actual content

How it works

1. Microprocessor generates a memory address.
2. **Index field** → used to select a **set in cache**.
3. **Tag field** → compared **associatively with all tags in the set**.
 - **Hit**: data is read from cache
 - **Miss**: data is fetched from main memory and placed in the cache set

Key point:

- Larger set size → better **hit ratio** (more words with same index can be stored)
- Drawback → more **hardware for tag comparison** → higher cost

Example (Set size = 2)

- Each cache word stores **2 memory words + 2 tags**
- Each word = 16 bits, each tag = 4 bits
- Word length = $2 \times (16 + 4) = 40$ bits
- Cache index = 8 bits → 256 sets
- Total cache size = 256×40 bits → can store **512 memory words**

Operation:

- Memory addresses 000 & 200 → stored in set 00
- Memory addresses 101 & 201 → stored in set 01
- CPU accesses memory → uses **index** to find set → compares **tags** → hit/miss

Cache Write Policies

1. Write-Back

- CPU writes to cache → mark word as **“dirty”**
- Dirty word replaced → write back to main memory **only when replaced**
- **Advantage:** fewer writes to main memory → faster

2. Write-Through

- CPU writes to cache → **main memory updated immediately**
- **Advantage:** main memory always valid → good for **multiprocessor systems**
- **Disadvantage:** more writes → slower

Cache Valid Bit

- Each cache block has a **valid bit**
- Valid bit = 0 → block holds **invalid data** → can be overwritten
- Valid bit = 1 → block holds **valid data**

On-Chip Cache Examples

- **Motorola 68020:** direct-mapped **instruction cache** only
- **Intel Pentium II:** 2-level cache:
 - **L1 cache:** inside CPU → small, separate instruction & data caches
 - **L2 cache:** outside CPU → larger, supplies L1 if L1 misses
- **Benefit:** improves performance by reducing main memory access
- **Set-associative:** index selects **set**, tag compared within set → reduces misses vs direct mapping